

Web Crawler Project

Demo: demo_video.mp4 (included in submission)

This is a web crawler and search engine built with Python (Flask + SQLite). It crawls web pages using BFS traversal with back pressure controls, and provides real-time search over the indexed content. The crawling logic uses only Python's standard library (no BeautifulSoup, no Scrapy, no requests). SQLite runs in WAL mode, which allows search to work while crawls are still running.

The project was built using Claude Code. It runs entirely on localhost and can be started with `python app.py`.

Pages

1. Crawler (Home Page)

Users create new crawl jobs by entering an origin URL and depth (k). There are also optional parameters: hit rate (requests per second), max URLs to crawl, and queue capacity. Clicking "Start Crawl" spawns a new thread that begins BFS traversal from the origin URL. The page shows live system stats (total jobs, pages indexed, unique words, active crawlers) and a table of recent crawl jobs with links to each job's status page.

2. Crawler Status

This page shows live monitoring for each crawl. It polls the server every 2 seconds and displays the current status (running, paused, stopped, or completed), pages crawled, queue size, and max depth. There is a detailed info table with the crawler ID, origin URL, hit rate, max URLs, and queue capacity. A live logs section shows color-coded entries (info, warning, error) as they come in. Users can pause, resume, or stop the crawl using the control buttons. When the queue fills up, back pressure warnings show up in the logs.

3. Search

Users type one or more keywords and get results as triples: (relevant_url, origin_url, depth), along with a relevance score and the words that matched. Results can be sorted by relevance, frequency, or depth. There is also an "I'm Feeling Lucky" button that picks a random indexed word. Pagination is available for large result sets.

Crawler Job (Major Component)

This is the core of the project. It takes a URL and depth, along with settings for hit rate, max URLs, and queue capacity.

When a crawl starts, a unique ID is generated from the epoch timestamp and thread ID (format: [EpochTime_ThreadID]). The crawler uses a bounded queue `Queue(maxsize=max_queue)` for BFS traversal. When the queue is full, new URLs are skipped and a warning is logged.

For each URL in the queue, the crawler:

1. Checks if the URL was already visited (in-memory set + DB lookup)
2. Waits based on the rate limit between requests (in 0.1s chunks so pause/stop stay responsive)

3. Fetches the HTML with `urllib.request` (SSL fallback, 10s timeout, 10MB size limit)
4. Parses the HTML with `html.parser.HTMLParser` to extract the title, visible text, links, and word frequencies
5. Stores page metadata in the `pages` table and word-URL mappings in the `word_index` table
6. Adds discovered links at `depth+1` to the queue (if within `max_depth` and the queue is not full)

The crawler supports pause, resume, and stop through thread-safe `threading.Event` objects. When stopped, the remaining queue is saved to the database so the crawl can pick up where it left off later.

Search (Minor Component)

When a user types a query, each word is normalized and looked up in the `word_index` table:

- **Exact matches** get a 10x frequency bonus
- **Prefix matches** (via SQL `LIKE 'word%'`) get the base frequency score
- **Multi-word bonus** (+100 per matched word) pushes up results that match more query terms
- **Depth penalty** (-5 per depth level) favors shallower pages

Results are sorted by the selected method (relevance, frequency, or depth) and paginated. Since the database runs in WAL mode, search queries never block the crawler, so users can search while crawls are still going.

Technical Details

Component	Details
Storage	SQLite with WAL mode, B-tree indexes on word lookups
Concurrency	Thread-per-crawl model, multiple crawls run at the same time
Back Pressure	Bounded queue + configurable rate limiting + max URL cap
Resumability	Queue state saved on stop, visited URLs loaded from DB on resume
Testing	54 unit tests covering HTML parser, search service, and crawler job
Dependencies	Flask is the only external dependency; all crawling uses Python stdlib